

VERITRUST

IMPLEMENTATION HANDBOOK | 11 AUGUST 2026

Learning Module Architecture Blueprint

A plain-language, repository-grounded plan for courses, lessons, progress, tests, assignments, certifications, administration, scalability and safe production release.

- Current implementation mapped to DB.txt
- 50 concrete gaps and failure journeys
- Corrected end-to-end flows and state machines
- Ordered tickets, tests, release gates and owner worksheet

Source basis: deployed schema snapshot, repository code, user interface and verification tooling



How to use this handbook

Start here

Read sections 1 to 4 for the present state. Use sections 5 to 12 as the target design. Execute the roadmap in section 13 in order. The last pages are fillable and can become the release checklist.

- **Current** means the behavior found in this repository and DB.txt on 11 August 2026.
- **Target** means the recommended production behavior. It is not a claim that the code already does it.
- **P0** means correctness, privacy or tenant-safety work that should block a production learning release.
- A module here means a group of lessons. An assessment means a test. A credential means the certificate record that proves an award.

Table of contents

How to use this handbook	2
Table of contents	3
1. Executive decision	5
What is strong today	5
What must block release	5
2. Current system map	6
Current pages and JavaScript	6
Current server routes	6
3. Current end-to-end learner flow	7
Where learners struggle	7
4. Deep audit by subsystem	8
4.1 Catalog, courses and content	8
4.2 Lesson player, progress and labs	8
4.3 Tests and scoring	8
4.4 Certifications and credentials	9
4.5 Assignments, cohorts and administration	9
5. Correct target learner experience	10
Step-by-step target flow	10
6. Target architecture	11
Build it as a modular monolith first	11
Scale safely	11
7. Content and publication design	12
The immutability rule	12
Suggested data additions	12
8. Reliable assessment and test system	13
Attempt creation	13
Answer saving and submission	13
Question policies	13
Results and retakes	13
9. Certification and credential system	14
Eligibility contract	14
Issue safely	14
Credential experience	14
10. Assignments, cohorts, programs and administration	15
Assignment flow	15
Authoring and review	15
11. API, data and security contracts	16

Recommended v2 route groups	16
Contract rules	16
Migration order	16
12. UI, accessibility and quality	17
UI cleanup	17
Accessibility target	17
Testing pyramid	17
13. Ordered implementation roadmap	18
Delivery gates	18
First ten tickets	18
14. Release measurements and operations	19
Suggested service objectives	19
Operational dashboards	19
Incident runbooks	19
15. Interactive product decisions	20
Architecture decisions	20
16. Implementation and release checklist	21
Approval record	21
17. Repository evidence and standards	22
Repository evidence reviewed	22
External standards used as design guidance	22

1. Executive decision

Decision

Keep the existing database foundation, but rebuild the learning behavior around strict tenant boundaries, immutable published content, a reliable assessment state machine and a real certification eligibility engine. Start as a modular monolith. Split services only when load or team ownership proves the need.

The repository already contains more learning infrastructure than the current screens reveal. The main problem is not a missing database. The main problem is that important tables and rules are either not connected, only partly connected, or connected in a way that can produce wrong results.

What is strong today

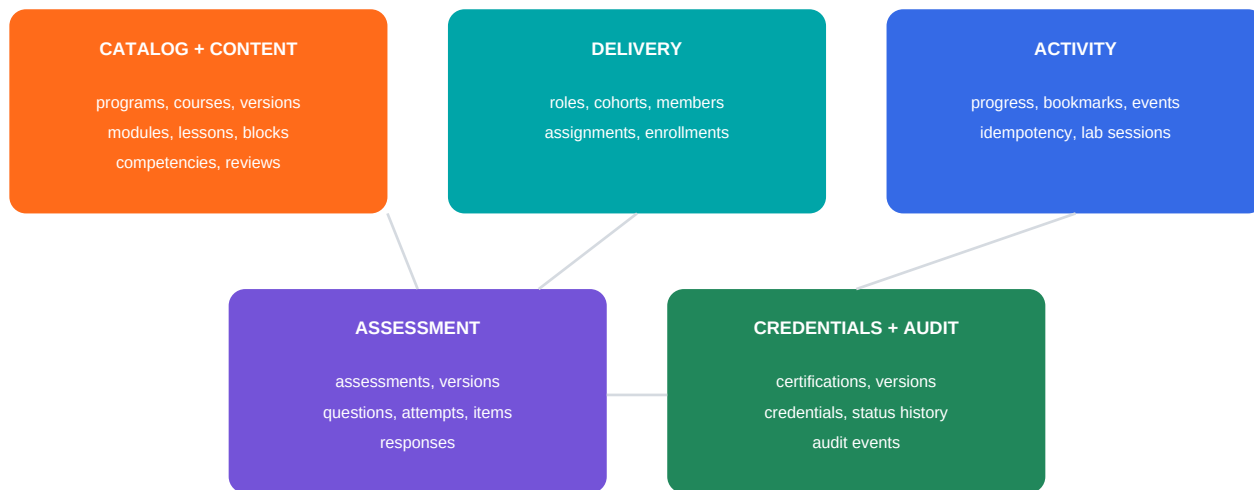
- 31 learning tables cover catalog, content versions, cohorts, assignments, progress, labs, tests, credentials and audit history.
- Published course, assessment and certification versions are represented separately from their parent objects.
- Server-side procedures already handle enrollment, events, response saving, grading and credential issuance.
- Correct answers are not returned to the browser during an active attempt.
- Idempotency receipts, forced row-level security, entitlement columns, rate limits and an HMAC preview gate are present.
- The repository static verification passes: 29 pages, 102 source files, security headers, metadata, local links and committed-secret rules were checked.

What must block release

Priority	Problem	Plain-language consequence
P0	Some learner reads are filtered by user, but not by the current organization	A learner can see records from another workspace and then fail when trying to update them.
P0	Multiple-choice questions are not rendered or scored correctly	A valid answer can receive zero marks.
P0	Answer saving can race with final submission	The learner can submit before the last answer reaches the server.
P0	Any passed course assessment may issue a credential	A practice quiz can incorrectly create a certificate.
P0	Published child content is still editable	A live lesson or answer key can change after a learner starts.
P0	Credential visibility defaults to public without a consent flow	Private achievement data may be exposed unintentionally.
P0	Assignment references are not fully validated	An enrollment can be linked to the wrong organization or learner target.

2. Current system map

The current product is a thin learning interface over a broad data model. The public catalog, course page, learner dashboard, lesson page, test page, certificate page and a small admin summary all call one learning API and repository layer.



31 learning tables already provide a strong base, but many are not connected to the product

Current pages and JavaScript

Area	Files	What it does now
Catalog	learning.html + learning.js	Loads a fixed catalog list and renders course cards.
Course	course.html + course.js	Shows basic course details and lessons; supports enrollment.
Lesson	lesson.html + lesson.js	Loads an enrolled lesson, renders blocks and marks completion.
Test	assessment.html + assessment.js	Starts an attempt, saves answers and submits for server grading.
Credentials	certificate.html + certificate.js	Lists a learner credential or verifies a public code.
Admin	learning-admin.html + learning-admin.js	Shows six summary metrics only.
Access	learning-access.html + middleware.ts	Uses a signed preview cookie while the module is gated.
Client API	assets/js/core/learning-api.js	Wraps the current GET, POST and PUT routes and creates idempotency keys.

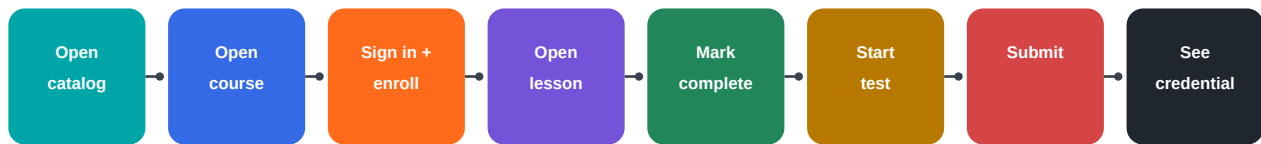
Current server routes

Type	Routes currently exposed	Important missing routes
Catalog	catalog, course by slug	pagination, programs, search facets, recommendations
Learner	me, enroll, enrollment, lesson, event	resume, bookmarks, labs, program progress, bulk sync
Test	start, attempt, save response, submit	history, review, manual grading, accommodations, proctoring
Credential	list and public verification	detail, privacy, PDF, badge, share, renew
Admin	summary	authoring, review, publish, cohorts, assignments, analytics and export

Reality check

The admin screen describes publishing, question editing, bulk assignment and exports, but those operations do not exist in the current API or interface. They must be treated as planned work, not finished capability.

3. Current end-to-end learner flow



This is the intended current path. Several links in it are incomplete or unsafe.

Where learners struggle

Journey	Root cause	What the person experiences
Preview lesson	A course links to a preview lesson without an enrollment, but the lesson page immediately asks the API for an enrollment ID.	The preview link fails instead of opening a read-only lesson.
Signed-out enrollment	The course page shows Enroll, but it has no sign-in redirect or clear call to action.	The learner sees an API error.
Rich lesson	Only checklist blocks have a special renderer. Scenario, lab, media and knowledge-check blocks become plain text.	The course feels unfinished and important interactions disappear.
Resume learning	Last position, bookmarks and lab sessions exist in the database but are not wired to the UI.	A returning learner must find the place again.
Test answer	Saving happens on change, and submission does not wait for pending saves.	The last answer may be omitted.
Expired test	An abandoned in-progress attempt is not consistently changed to expired before another attempt starts.	Attempt limits can be bypassed and history becomes unclear.
Certificate	The client loads only the first credential page and ignores the next cursor.	Older certificates disappear from the interface.
Workspace switch	Some repository reads use only user_id while service credentials bypass normal row policies.	Old-workspace records can appear, then writes fail.

4. Deep audit by subsystem

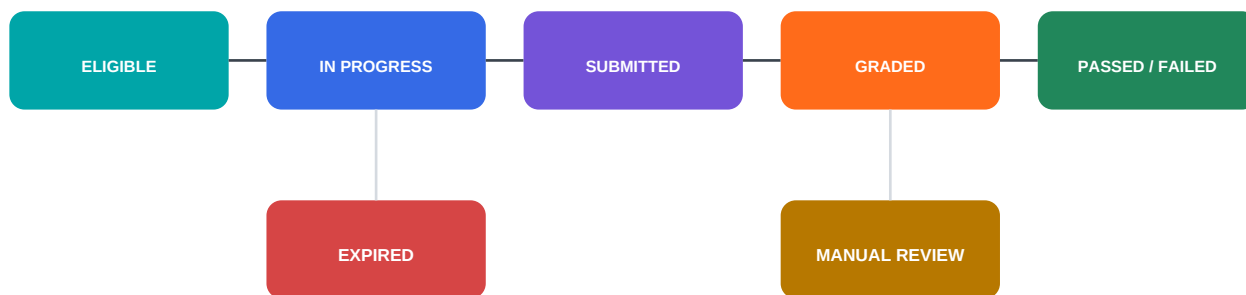
4.1 Catalog, courses and content

- Catalog returns a fixed maximum of 24 courses. There is no cursor, page control, program view, category facet or recommendation logic.
- Stored learning outcomes, prerequisites, labs and certificate rules are not fully explained on the course page.
- Prerequisites exist as JSON but are not enforced before enrollment, lesson access, final assessment or credential issuance.
- Course completion is based mainly on completed lesson count. Labs, required tests and prerequisites are not part of one deterministic rule.
- Mutable lessons define the denominator. Adding a lesson later can change progress for an existing enrollment.

4.2 Lesson player, progress and labs

- The player needs previous and next navigation, a module outline, resume position, autosave status, bookmarks and a clear completion rule.
- Block renderers must exist for text, image, video, checklist, scenario, knowledge check, lab, download and callout. Unknown types must fail safely.
- `accessibility_label` is stored but ignored. Media needs captions, transcripts, keyboard control and text alternatives.
- `learning_lab_sessions`, `learning_bookmarks`, `block_completed`, `lab_completed` and `last_position` are currently mostly unused.
- Progress writes need monotonic rules: completed must never accidentally return to incomplete, and concurrent devices must not lose the newest position.

4.3 Tests and scoring



Every transition is server-controlled, timestamped, idempotent and auditable

Finding	Why it matters	Required fix
Multiple choice has no correct UI or scoring branch	It always behaves like text and can score zero	Render checkbox choices and implement set comparison with explicit partial-credit policy.
Save and submit are separate browser actions	The final answer can be missed	Keep a client save queue, bulk flush it, then submit with a server transaction and attempt revision.
Expired attempts are not finalized reliably	Unlimited abandoned attempts may be possible	Expire before start/read/submit and count starts according to the approved attempt policy.
Every question is selected	No controlled test form or competency coverage	Create a server-side blueprint: pool, count, difficulty and competency distribution.
Short text uses exact normalized equality	Correct explanations can be marked wrong	Use manual grading, approved patterns or a rubric. Do not use a language model as the final authority without human review.
Timer is browser-led	Clock drift and disconnected clients are confusing	Display server deadline, re-sync, warn, and perform server-controlled expiry or auto-submit.

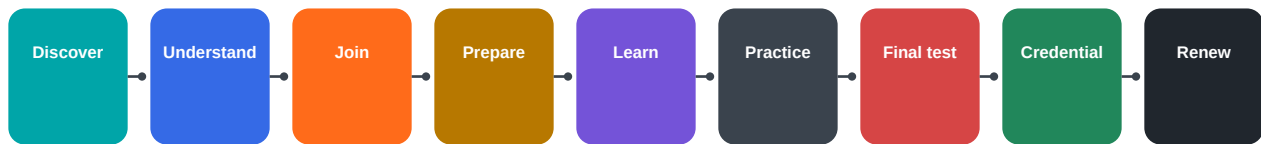
4.4 Certifications and credentials

- The submit procedure can issue a credential after any passed assessment linked to a course, including practice or lab tests.
- Certification requirements are stored, but they are not evaluated as a complete eligibility contract.
- Credential access entitlement is not enforced in the automatic issuance path.
- `public_visible` defaults to true, but the product has no explicit learner consent screen.
- Expiry is a date, but no reliable job or read-time effective-status rule changes a VALID credential after that date.
- Credential uniqueness does not include organization even though credentials store `org_id`. Cross-organization completion can reuse the wrong record.
- There is no certificate PDF, standards-based badge, evidence page, privacy control, revocation timeline or renewal flow.

4.5 Assignments, cohorts and administration

- Programs, competencies, roles, cohorts, assignment targets and content reviews exist in the schema but have no complete product workflow.
- `assignment_id` can reach enrollment without a full same-organization, same-course, target-person and due-state check.
- The assignment table permits both target columns or neither target column. Enforce exactly one target, or use a normalized target table.
- Seat limit, assignment limit, export access and proctored exam access are defined in plans but are not fully enforced.
- There is no author-review-approve-publish screen, version diff, scheduled release, rollback, due reminder or manager report.

5. Correct target learner experience



One visible journey, backed by strict server rules and resumable state.

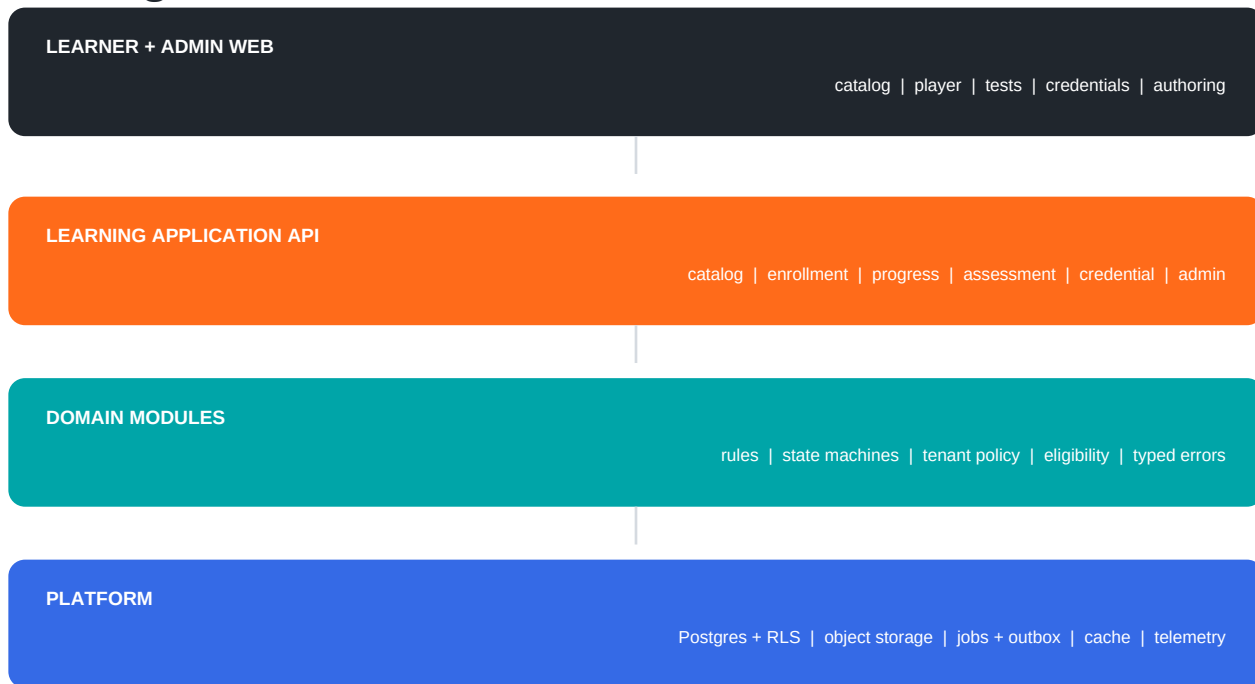
Step-by-step target flow

Stage	Required behavior
1. Discover	Search, filter and page through courses and programs. The catalog endpoint returns a cursor and stable sort order.
2. Understand	Course detail clearly shows level, outcomes, prerequisites, duration, modules, labs, final test and certificate requirements.
3. Join	A signed-out person gets a sign-in return path. The server checks tenant membership, plan entitlement, seat limit and prerequisites, then enrolls once.
4. Prepare	A readiness page explains missing prerequisites, accommodations, deadline and required activities.
5. Learn	The player opens a publication snapshot, resumes the last position, autosaves safely and offers accessible block renderers.
6. Practice	Practice checks give feedback but never issue credentials. Labs have their own session and completion state.
7. Final test	The server creates an immutable question snapshot, enforces time and attempts, saves answers with revisions and grades under one rule.
8. Complete	A single eligibility engine evaluates lessons, labs, final tests, prerequisites and any manual review.
9. Receive credential	Issue once for the user, organization and certification version. Default it to private until the learner chooses to share.
10. Maintain	Show expiry, renewal steps, status history and revocation reason. Verification always reports effective status.

One source of truth

The same eligibility function must drive the dashboard, course completion, admin reports and credential issuance. Do not duplicate completion logic in JavaScript, SQL procedures and background jobs.

6. Target architecture



Build it as a modular monolith first

A modular monolith means one deployable application with strict internal boundaries. It is easier to operate than early microservices, while still allowing a busy module to be separated later.

Module	Owns	Must not do
Catalog and content	programs, courses, versions, modules, lessons, assets, publication manifests	modify a published snapshot
Enrollment and assignments	membership, seats, prerequisites, cohorts, assignments, enrollment state	issue a credential
Progress and labs	resume, bookmarks, block events, lesson completion, lab sessions	decide final certification alone
Assessment	attempt policy, snapshots, response revisions, timers, grading, review	expose answer keys to the client
Eligibility and credentials	completion rules, evidence, issuance, privacy, status, renewal	guess from a browser progress percentage
Admin and authoring	drafts, review, publish, cohorts, assignments, reporting	bypass role, tenant or plan checks
Jobs and notifications	outbox, reminders, expiry, aggregates, retries	perform untracked side effects

Scale safely

- Use read models for dashboard and admin summaries instead of expensive live joins on every request.
- Put static published media in object storage behind a CDN. Store asset metadata and signed access policy in Postgres.
- Write domain events and an outbox record in the same database transaction. A worker sends notifications and updates analytics with retries.
- Partition high-volume `learning_events` and audit data by time only when measured volume requires it. Add indexes from real query plans.
- Cache public catalog data by publication version. Never cache tenant-specific enrollment or credential data under a shared key.
- Define service-level objectives: API availability, save latency, submission completion, job delay and public verification latency.

7. Content and publication design



Published content and all descendants are sealed; change means clone a new draft version.

The immutability rule

The current lock protects the top-level version row, but child modules, lessons, blocks and question revisions can still change. That is unsafe because a learner can start one reality and be graded against another.

```

publish(course_version_id):
  validate complete tree and assets
  calculate canonical manifest_hash
  seal course version + every child revision
  record actor, timestamp and review evidence
  emit learning.course_version.published

edit published content:
  clone to a new draft version
  keep the old publication unchanged
  
```

Suggested data additions

Addition	Purpose
learning_publication_manifests	Immutable tree hash, published timestamp, actor and dependency versions.
content_revision_id on progress	Pins an enrollment and its progress to the exact content learners saw.
learning_assets	File metadata, captions, transcript, alt text, checksum and storage key.
validated completion rule	A versioned rule describing required lessons, labs and final assessments.
review approvals	Named reviewer, decision, comment and separation-of-duty evidence.

8. Reliable assessment and test system

Attempt creation

- Check tenant, enrollment, prerequisites, assessment availability, accommodations, attempt count and cooldown on the server.
- Finalize expired attempts before deciding whether another start is allowed.
- Select questions with a versioned blueprint. Save an immutable snapshot of prompt, choices, points, rubric and answer-key reference.
- Return safe client content only. Store the server deadline and attempt revision.

Answer saving and submission

```
Browser: edit answer -> local queue -> PUT response with attempt_revision
Server: validate ownership + state + deadline -> upsert revision -> return saved_at
Browser: Submit -> stop edits -> flush every pending save -> POST submit
Server transaction: lock attempt -> verify revision -> finalize -> grade -> write result/event/outbox
Browser: show the server result; retry the same idempotency key after a network failure
```

Question policies

Type	Rendering	Scoring policy
Single choice	Radio group	Exact selected option.
Multiple choice	Checkbox group	Exact set by default; partial credit only when explicitly configured.
True or false	Two radio choices	Exact value.
Numeric	Number field with units	Tolerance and units defined in the revision.
Short text	Text field	Approved normalization/pattern for simple facts; otherwise manual rubric.
Long answer	Large text field	Manual grading task with rubric and second-review option.
Practical lab	Launch and evidence controls	Signed lab result or human review; never browser self-report alone.

Results and retakes

- Store auto score, manual score, final score, pass rule and grading version separately.
- Define review policy: immediately show explanations, show them after the test closes, or never show protected items.
- Show attempt history, remaining attempts, next eligible time and a clear appeal path.
- Proctoring must be optional by plan and assessment policy, privacy reviewed, and never silently activated.

9. Certification and credential system



A passed practice quiz is evidence, not automatic proof of certification.

Eligibility contract

```

eligible =
  enrollment belongs to current organization
  AND required lesson snapshot is complete
  AND required labs are complete
  AND required final assessments are passed
  AND prerequisites are satisfied
  AND manual reviews are approved
  AND organization plan allows certificates
  AND certification version is active
  
```

Issue safely

- Use a unique key including organization_id, user_id and certification_version_id unless the business deliberately defines a global credential.
- Save an evidence snapshot so future verification can explain exactly why the credential was issued.
- Issue in an idempotent transaction and write credential history, audit event and outbox event together.
- Default public visibility to false. Ask for explicit consent before showing the learner name or evidence publicly.
- Compute effective status at read time and run a scheduled expiry job. Expired must override VALID after expires_at.

Credential experience

Screen	Must show
Learner wallet	All credentials with cursor pagination, status, expiry, renewal and privacy control.
Credential detail	Award, issuer, issued date, evidence, skills, status history and download/share actions.
Public verification	Minimum consented fields, effective status, issuer, award and tamper-resistant identifier.
Admin	Issue/revoke/suspend/renew controls with reason, two-person review where required and full audit.
Portable output	Accessible PDF plus optional Open Badges 3.0-compatible credential when interoperability is needed.

10. Assignments, cohorts, programs and administration

Assignment flow



A target is either a person or cohort - never both and never neither.

Area	Required implementation
Cohorts	Create, edit, archive, add/remove members, import safely and show membership history.
Assignments	Validate same org and course, exactly one target, due date, seat and assignment limits; create idempotently.
Programs	Order course requirements, enroll learners, calculate program progress and issue program credentials separately.
Competencies	Map versioned course outcomes and test evidence to a pinned competency framework version.
Manager view	Assigned, not started, in progress, overdue, completed, failed and expiring - with filters and export entitlement.
Notifications	Assignment, due soon, overdue, completion, credential and expiry messages through outbox-driven jobs.

Authoring and review

- Author creates a draft version, uses structured blocks and supplies accessibility metadata.
- Validator checks missing titles, broken assets, empty outcomes, invalid rules, answer keys and estimated duration.
- Reviewer sees a version diff and approves or requests changes. The author cannot silently change an approved revision.
- Publisher schedules or publishes a sealed manifest. Rollback changes which version is active; it never edits history.
- Every admin mutation uses role, tenant and plan checks and creates an audit event with request ID.

11. API, data and security contracts

Recommended v2 route groups

Group	Examples
Catalog	GET /v2/learning/catalog?cursor=...; GET /courses/:slug; GET /programs/:slug
Learner	GET /me/dashboard; POST /enrollments; GET /enrollments/:id/resume; PUT /progress; PUT /bookmarks
Labs	POST /lab-sessions; PUT /lab-sessions/:id; POST /lab-sessions/:id/submit
Tests	POST /attempts; PUT /attempts/:id/responses:sync; POST /attempts/:id/submit; GET /attempts/:id/result
Credentials	GET /credentials; PATCH /credentials/:id/privacy; GET /credentials/:id/pdf; GET /verify/:code
Admin	courses/versions/reviews/publish; cohorts; assignments; grading; credentials/status; analytics/exports

Contract rules

- Every organization-scoped repository method receives org_id explicitly and includes it in every read and write predicate.
- Entitlement lookup is fail-closed: missing plan data denies the paid capability and returns a typed error.
- Use RFC 9457-style problem details with stable error codes such as LEARNING_PREREQUISITE_MISSING, ATTEMPT_EXPIRED and SEAT_LIMIT_REACHED.
- Every mutation accepts an idempotency key. Updates to mutable records also use a revision or ETag for optimistic concurrency.
- Cursor pagination compares the complete stable order tuple, for example (created_at, id), so equal timestamps do not skip records.
- Service-role database access never replaces application authorization. Row-level security remains defense in depth.

Migration order

Migration	Change	Safe rollout
M1	Add/repair tenant indexes and organization-aware credential uniqueness	Detect conflicts, backfill, then enforce.
M2	Add assignment target constraint and cross-reference validation	Report invalid rows before adding constraints.
M3	Add publication manifest and content snapshot references	Snapshot every currently published version first.
M4	Add attempt snapshot, response revision and manual-grading task	Dual-write, verify, then switch grading.
M5	Add credential evidence and private-by-default visibility	Preserve existing public links only under an approved migration policy.
M6	Add outbox and read models	Backfill aggregates and compare with live queries before cutover.

12. UI, accessibility and quality

UI cleanup

learning.css is about 32 KB for a small set of screens and contains repeated mobile breakpoints and later override layers. This makes behavior difficult to predict. Replace stacked patches with one small design system and page components.

Component	States that must be designed
Page shell	desktop, tablet, mobile, narrow phone, print
Data panel	loading, empty, success, partial, error, retry, offline
Save control	unsaved, saving, saved time, conflict, failed, retry
Course card	public, enrolled, assigned, complete, locked, archived
Lesson block	supported, loading media, unavailable media, unknown safe fallback
Test	ready, active, reconnecting, expired, submitted, grading, graded
Credential	private, public, valid, expired, suspended, revoked, renewing

Accessibility target

- Target WCAG 2.2 AA for the complete learning process, not only individual screens.
- Use semantic headings, labelled controls, visible focus, predictable keyboard order, status announcements and no keyboard traps.
- Provide captions and transcripts for video, alt text for meaningful images and accessible names from accessibility_label.
- Allow timing accommodations. Warn before expiry and do not rely only on color or animation.
- Test zoom, reflow, touch target size, screen readers, reduced motion and high contrast with real user journeys.

Testing pyramid

Level	Must cover
Unit	validators, eligibility rules, attempt policy, scoring types, status calculation, cursor encoding
Database	RPC contracts, constraints, immutability, RLS and organization isolation
Integration	enroll, progress, attempt lifecycle, grading, evidence, credential issuance, expiry and jobs
End to end	catalog -> sign in -> enroll -> lesson -> lab -> final test -> private credential -> consented verification
Adversarial	cross-org IDs, replayed mutations, stale revisions, altered timers, enumeration and answer-key leakage
Experience	accessibility, visual regression, responsive layout, browser support and slow/offline networks
Scale	catalog burst, autosave burst, mass assignment, deadline submission spike, notification retries

Current verification gap

The repository static check passes, but it does not prove learning behavior. There are no complete unit, database, integration or end-to-end tests for the learner journey. Add them before calling the module production-ready.

13. Ordered implementation roadmap



Do not build admin polish before correctness, tenancy and published-content safety are fixed

Delivery gates

Gate	Build	Exit evidence
D0 - Truth and contracts	Write state tables, error codes, completion rules and API contracts. Add a behavioral test harness.	Approved diagrams, fixtures and failing tests for every P0.
D1 - Tenant safety	Add org filters to all reads/writes, fail-closed entitlements, credential uniqueness and assignment validation.	Automated two-org isolation suite passes.
D2 - Immutable publication	Seal descendants, create manifests, snapshot content/questions and clone new drafts.	Published data cannot change; active attempts grade against snapshots.
D3 - Learner player	Repair preview/sign-in, create renderer registry, navigation, resume, bookmarks, labs and progress rules.	Course and lesson E2E passes on phone, desktop and keyboard.
D4 - Test reliability	Fix multiple choice, save queue, bulk flush, expiry, limits, blueprint, results and manual review.	No lost-answer race; every question type has golden tests.
D5 - Credentials	Build eligibility evidence, entitlement check, private default, expiry, wallet pagination, PDF and verification.	Practice tests cannot issue; privacy and lifecycle tests pass.
D6 - Programs and assignments	Wire cohorts, one-target assignments, program progress, due states, reminders and read models.	Mass assignment load and quota tests pass.
D7 - Admin and authoring	Draft/review/publish, diff, roles, grading queue, analytics, export and audit experience.	Least-privilege matrix and content lifecycle tests pass.
D8 - Production release	Accessibility, observability, SLOs, load, migration rehearsal, canary, rollback and support runbook.	Release checklist signed; canary metrics remain within limits.

First ten tickets

ID	Priority	Acceptance summary
LRN-001	P0	Enforce organization scope in me, enrollment, attempt and credential reads; add two-org tests.
LRN-002	P0	Repair preview lesson flow and signed-out enrollment return path.
LRN-003	P0	Implement multiple-choice rendering/scoring and save-then-submit synchronization.
LRN-004	P0	Finalize expired attempts and enforce the approved maximum-attempt policy.
LRN-005	P0	Create one eligibility engine; prevent practice/lab tests from issuing credentials.
LRN-006	P0	Seal published descendants and snapshot active-attempt questions.
LRN-007	P0	Validate assignment organization, course, target, state, due date and limits.
LRN-008	P0	Make credentials organization-safe, private by default and expiry-aware.
LRN-009	P0	Add typed API problems, request IDs and fail-closed entitlement handling.
LRN-010	P0	Build unit, database, integration and E2E test lanes in CI.

14. Release measurements and operations

Suggested service objectives

Measure	Initial target	Alert when
Progress save	p95 under 500 ms; 99.9% successful excluding client validation	failure or conflict rate rises
Final submission	99.99% accepted exactly once or safely retryable	unknown submission state exists
Public verification	p95 under 400 ms with effective status	stale or unavailable verification
Assignment job	95% processed within 5 minutes	queue age exceeds objective
Credential issuance	99% within 60 seconds of eligibility	eligible learner lacks credential
Accessibility	No critical/serious automated issue plus manual journey pass	regression blocks a complete process

Operational dashboards

- Learner funnel: course viewed, enrollment started, enrolled, first lesson, active, completed, final attempted, passed, credential issued.
- Reliability: save failures, stale revisions, submission retries, expired attempts, grading backlog and job queue age.
- Integrity: cross-tenant denials, rule-version mismatch, duplicate issuance prevention and publication-hash failures.
- Content: lesson abandonment, question difficulty, distractor quality, competency coverage and accessibility defects.
- Business: active learners, completion by cohort, overdue assignments, seat use, renewals and consented shares.

Incident runbooks

Incident	First safe action
Answers not saving	Stop new starts if needed, preserve local queues, inspect request IDs and allow idempotent replay.
Wrong score	Suspend affected issuance, preserve snapshots, identify grading version and run deterministic regrade.
Wrong credential	Suspend rather than delete, retain evidence and history, notify affected users and correct the rule.
Cross-tenant exposure	Disable affected route, preserve audit evidence, rotate access if needed and follow the privacy response plan.
Bad publication	Switch active version to the last safe immutable version; do not rewrite completed learner history.

15. Interactive product decisions

Fill these fields before implementation. Blank answers indicate a product decision that can change the database or API behavior.

Primary learner audience

Expected active learners

Peak concurrent test takers

Required languages

Course completion rule

Default maximum attempts

Retake cooldown

Manual grading owner/SLA

Credential validity period

Public sharing consent wording

Accessibility conformance owner

Production release owner

Architecture decisions

- ☐ Credential scope is organization-specific
- ☐ Practice assessments never issue credentials
- ☐ Published descendants are immutable
- ☐ Credentials are private until explicit consent
- ☐ Final submission flushes pending answer saves
- ☐ Tenant checks are required even with service-role access
- ☐ WCAG 2.2 AA is the target
- ☐ Start with a modular monolith

16. Implementation and release checklist

- ☐ P0 tenant reads and writes are scoped to current organization
- ☐ Entitlements fail closed and every paid capability is mapped
- ☐ Preview, sign-in return and enrollment flows pass E2E
- ☐ Published content and question snapshots cannot be edited
- ☐ Every lesson block has a safe accessible renderer
- ☐ Resume, bookmarks, labs and progress conflict handling work
- ☐ Every question type has rendering and grading golden tests
- ☐ Submit waits for all saves and is exactly-once safe
- ☐ Expiry, attempt count, cooldown and accommodations are server-enforced
- ☐ Only the eligibility engine can trigger a credential
- ☐ Credential evidence, privacy, expiry and status history work
- ☐ Pagination uses the complete stable sort tuple
- ☐ Assignment target and same-org rules are database-enforced
- ☐ Admin roles, plan limits and audit events are tested
- ☐ Outbox retries are idempotent and monitored
- ☐ Unit, database, integration and learner E2E lanes pass
- ☐ Keyboard, screen reader, zoom, reflow and timing tests pass
- ☐ Load test includes a deadline submission spike
- ☐ Migration was rehearsed with conflict and rollback reports
- ☐ Canary, observability, incident and support runbooks are ready

Approval record

Product approval

Engineering approval

Security/privacy approval

Accessibility approval

Release date

17. Repository evidence and standards

Repository evidence reviewed

- DB.txt - deployed database schema snapshot, including 31 learning tables, functions, views, indexes, constraints and row-level security policies.
- api/learning.js; lib/learning/repository.js; lib/learning/validation.js; lib/learning-access.js; middleware.ts.
- learning.html; course.html; lesson.html; assessment.html; certificate.html; learning-admin.html; learning-access.html.
- assets/js/core/learning-api.js and learning page scripts; assets/css/pages/learning.css.
- scripts/verify.js; package.json; docs/ARCHITECTURE.md.

External standards used as design guidance

1. W3C Web Content Accessibility Guidelines (WCAG) 2.2 - accessibility requirements and testable success criteria.
<https://www.w3.org/TR/WCAG22/>
2. 1EdTech Open Badges - optional portable digital credential interoperability. <https://www.1edtech.org/standards/open-badges>
3. NIST NICE Framework current versions - optional common language for cybersecurity work roles, tasks, knowledge and skills. Pin the chosen component version in course metadata.
<https://www.nist.gov/itl/applied-cybersecurity/nice/nice-framework-resource-center/nice-framework-current-versions>
4. NIST SP 800-181 Rev. 1 - Workforce Framework for Cybersecurity (NICE Framework).
<https://www.nist.gov/publications/workforce-framework-cybersecurity-nice-framework>

Implementation starting point

Begin with LRN-001 through LRN-010. The safest first milestone is not a new admin screen; it is a tested tenant-safe learner path with immutable content, reliable answer submission and correct credential eligibility.

END OF HANDBOOK